K-Means Clustering

Using on the MNIST dataset

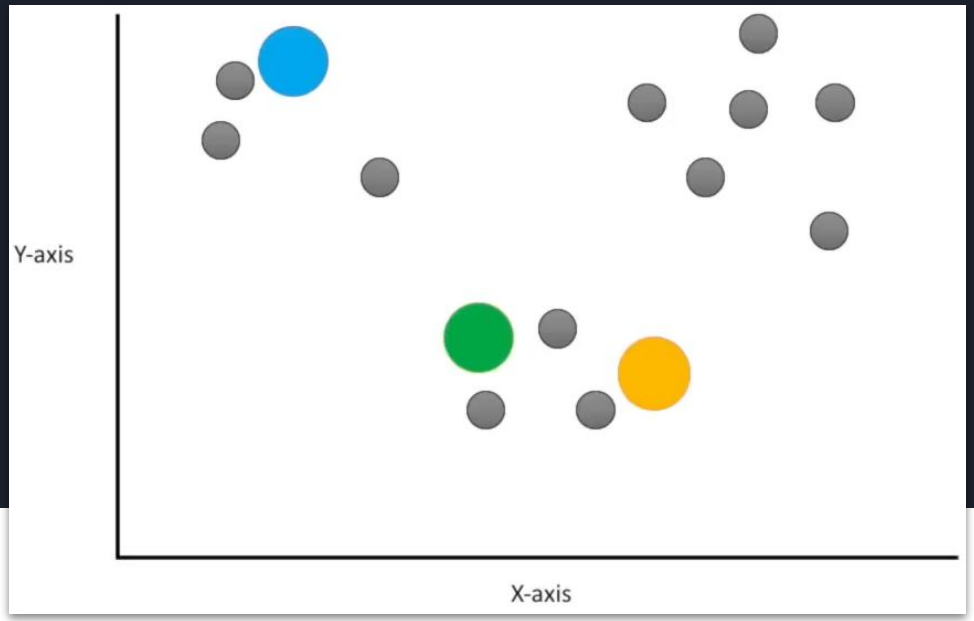
K-Means Overview

- K-means is a popular unsupervised machine learning algorithm used for partitioning a dataset into distinct, non-overlapping groups or clusters
- The goal of K-means is to group similar data points together and discover underlying patterns in the data
- K-means is computationally efficient and is often used in practice for large datasets
- K-means assumes that clusters are spherical and equally sized, which might not always be the case in real-world data

How k-means algorithm works (1/3)

Initialization:

Step 1: Choose the number of clusters (K) that you want to create. This is a hyperparameter, meaning you have to decide how many clusters you expect in your data.



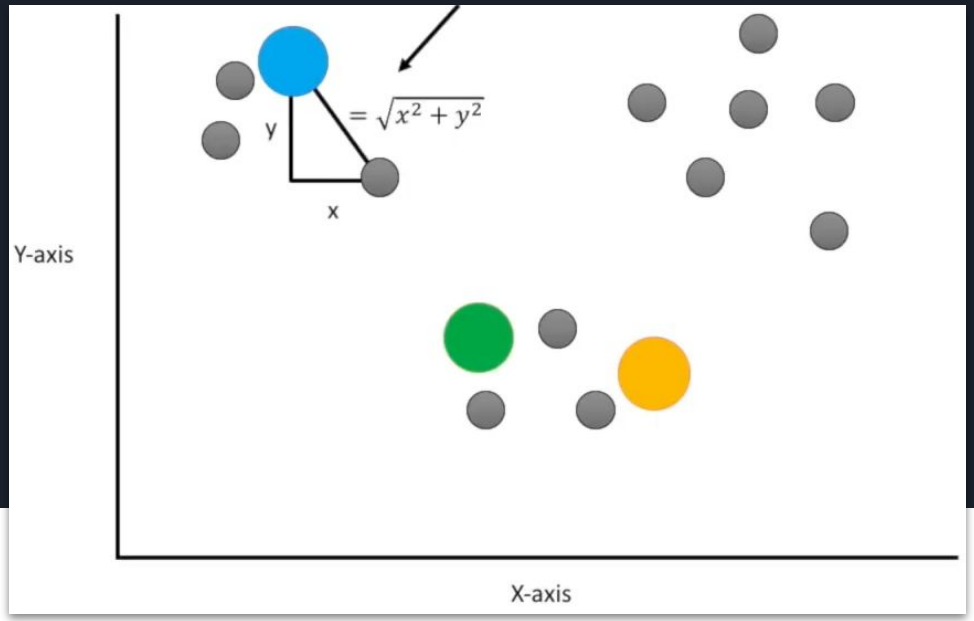
StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 2: Randomly initialize K cluster centroids. These centroids are the initial representative points for each cluster.

How k-means algorithm works (2/3)

Assignment:

Step 3: Assign each data point to the nearest cluster centroid. This is usually done using a distance metric, commonly the Euclidean distance.



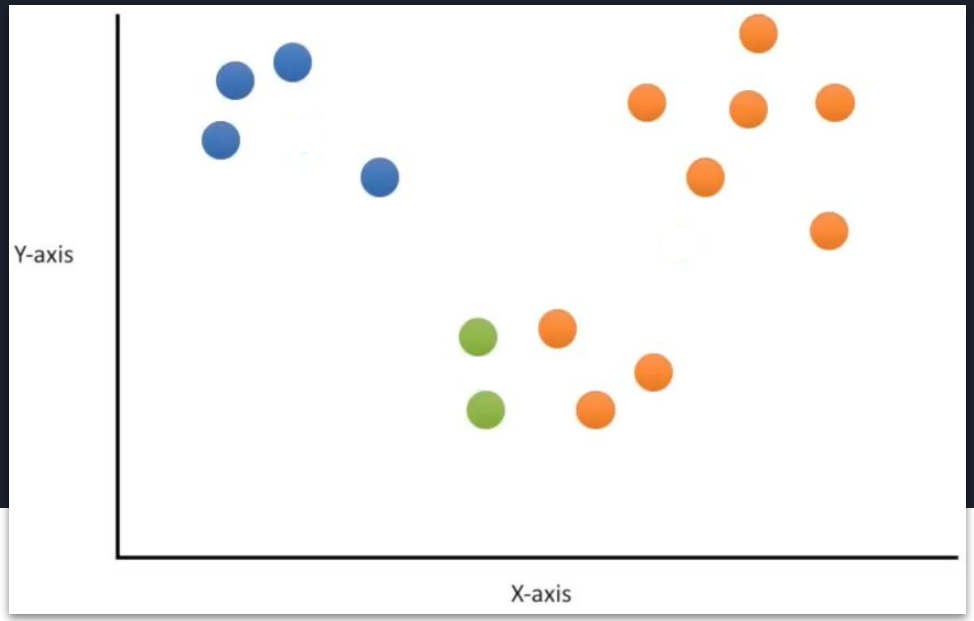
Step 4: All the points that are closer to a particular centroid are assigned to that cluster.

StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

How k-means algorithm works (2/3)

Assignment:

Step 3: Assign each data point to the nearest cluster centroid. This is usually done using a distance metric, commonly the Euclidean distance.



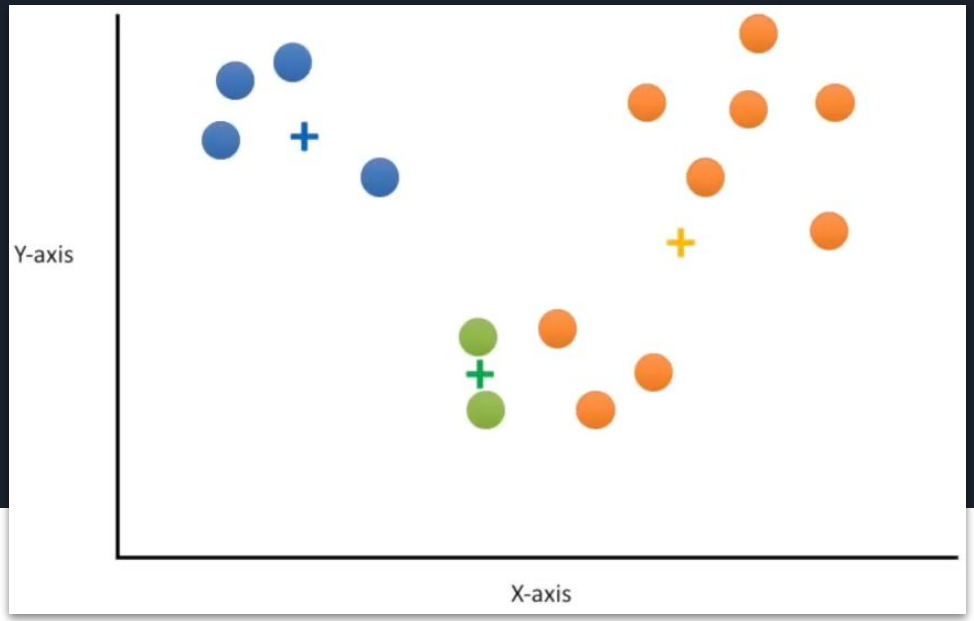
StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 4: All the points that are closer to a particular centroid are assigned to that cluster.

How k-means algorithm works (3/3)

Update:

Step 5: Recalculate the cluster centroids. The new centroids are the mean of all the data points assigned to that cluster in the previous step.



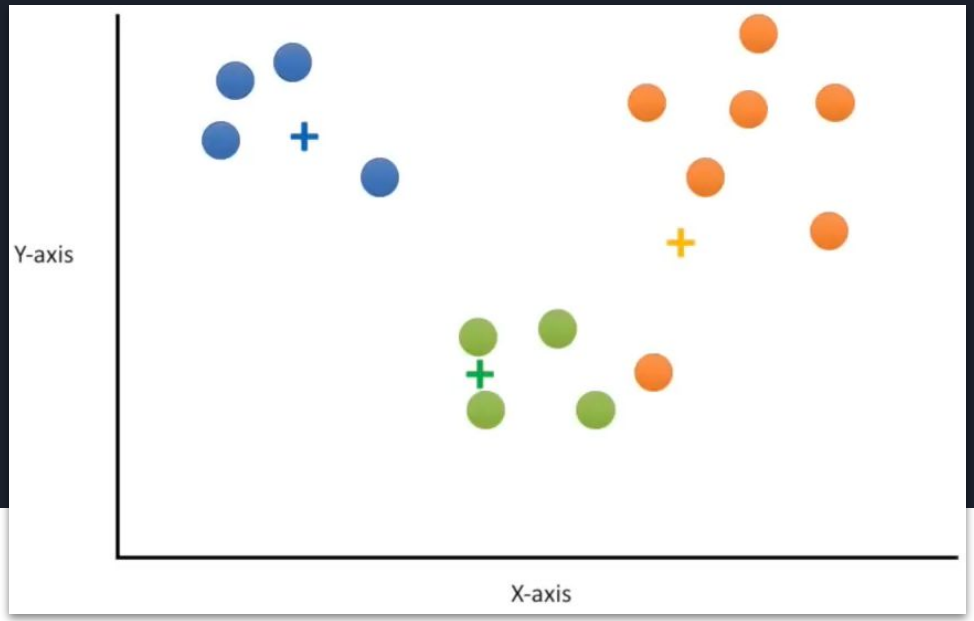
StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 6: Repeat steps 3-5 until the centroids no longer change significantly or a specified number of iterations is reached.

How k-means algorithm works (3/3)

Update:

Step 5: Recalculate the cluster centroids. The new centroids are the mean of all the data points assigned to that cluster in the previous step.



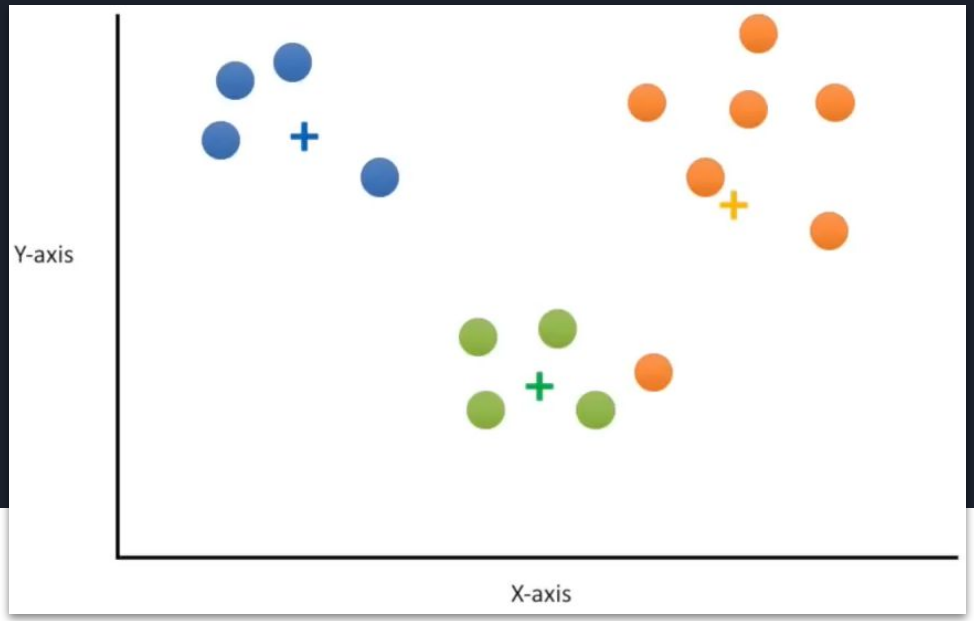
StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 6: Repeat steps 3-5 until the centroids no longer change significantly or a specified number of iterations is reached.

How k-means algorithm works (3/3)

Update:

Step 5: Recalculate the cluster centroids. The new centroids are the mean of all the data points assigned to that cluster in the previous step.



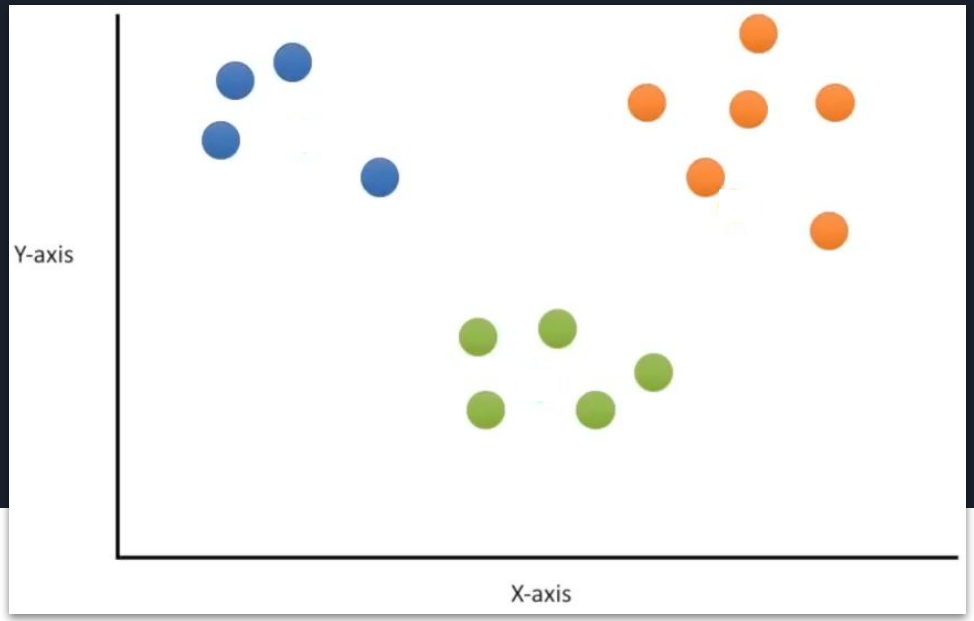
StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 6: Repeat steps 3-5 until the centroids no longer change significantly or a specified number of iterations is reached.

How k-means algorithm works (3/3)

Update:

Step 5: Recalculate the cluster centroids. The new centroids are the mean of all the data points assigned to that cluster in the previous step.



StatQuest: K-means clustering
https://youtu.be/4b5d3muPQmA

Step 6: Repeat steps 3-5 until the centroids no longer change significantly or a specified number of iterations is reached.

K-Means clustering on the MNIST dataset

- Parameters:
  - init = [k-means++, random, PCA-based]
    - k-means++: selects initial cluster centroids using sampling based on an empirical probability distribution of the points' contribution to the overall inertia
    - random: choose k samples at random from data for the initial centroids
    - PCA-based: selects the initial centroids based on the dataset's principal components
  - n_init=1
    - Number of times the k-means algorithm is run with different centroid seeds
  - random_state=0
    - Make the randomness of *init=k-means++* and *init=random* deterministic

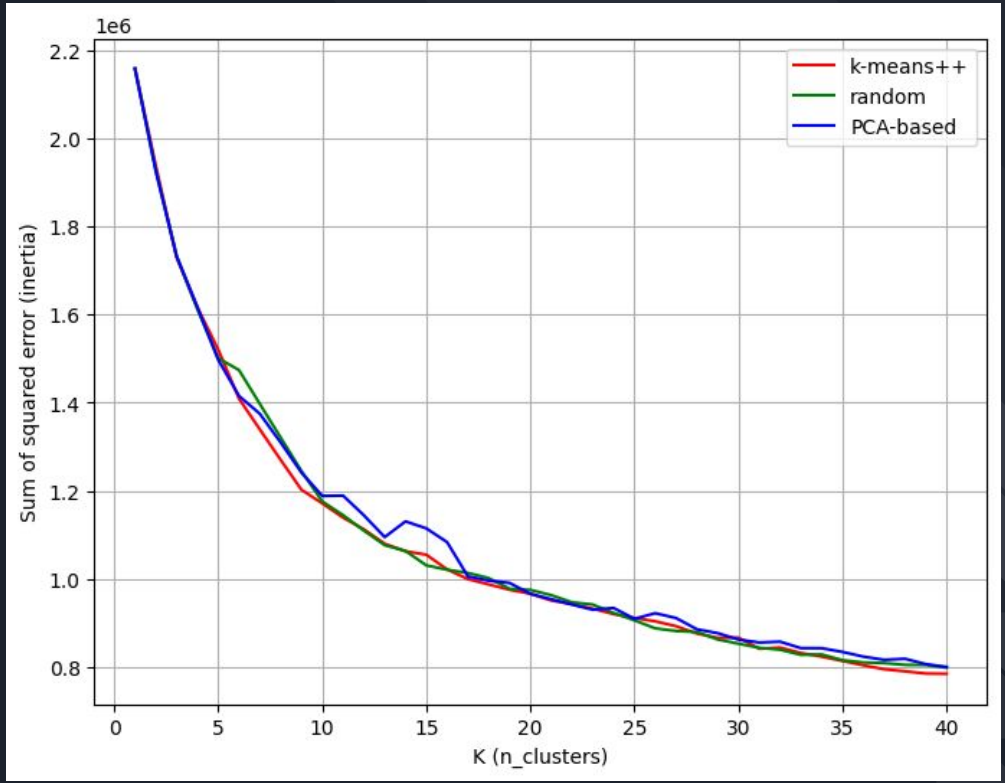
- Plot & compare within-cluster sum-of-squares (inertia) between each initialization method
- Plot the decision boundary & compare centroid positions between *init=k-means++* and *init=random*

Sum-of-squares error of each initialization method

n_clusters=[1...40]

Compute Time (seconds):

- k-means++ = 2.3
- random = 0.2
- PCA-based = 7.3



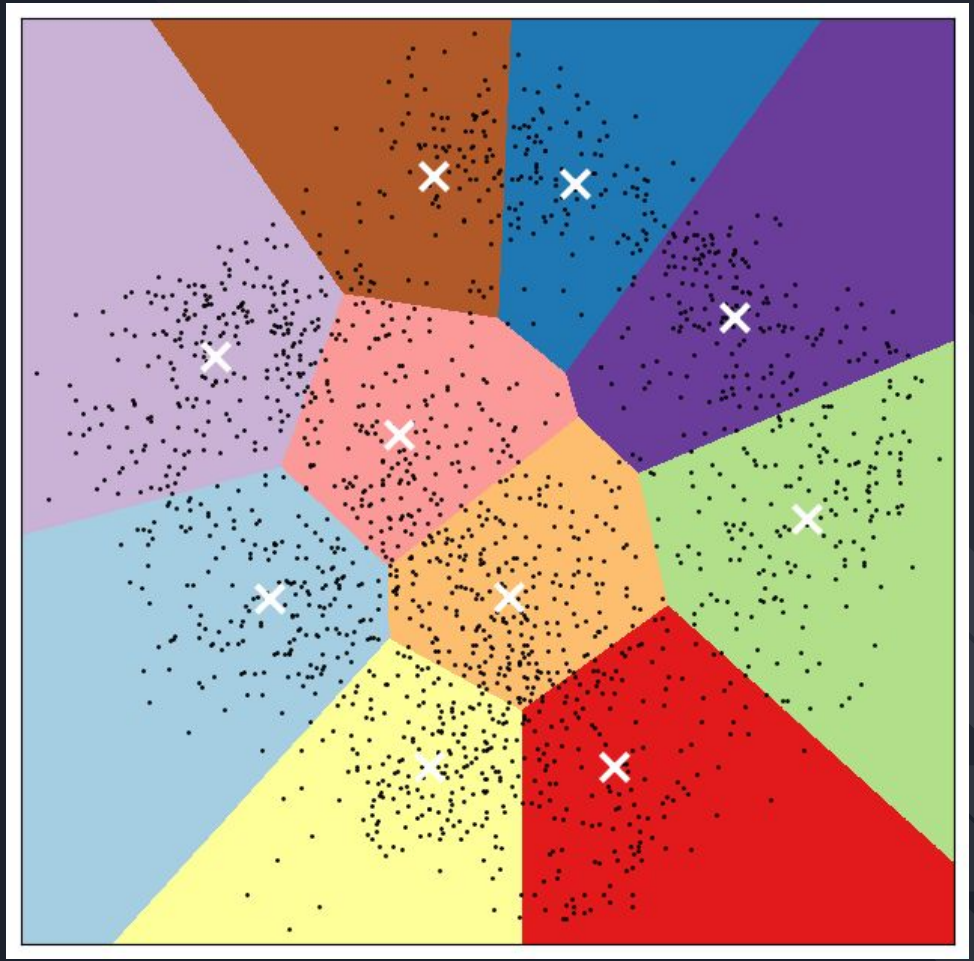
Decision Boundary with Centroids

Here, the MNIST dataset is reduced from 64D to 2D using Principal Component Analysis (PCA) for visualization

Parameters:

- init=k-means++
- n_clusters=10

Inertia: 57304.14



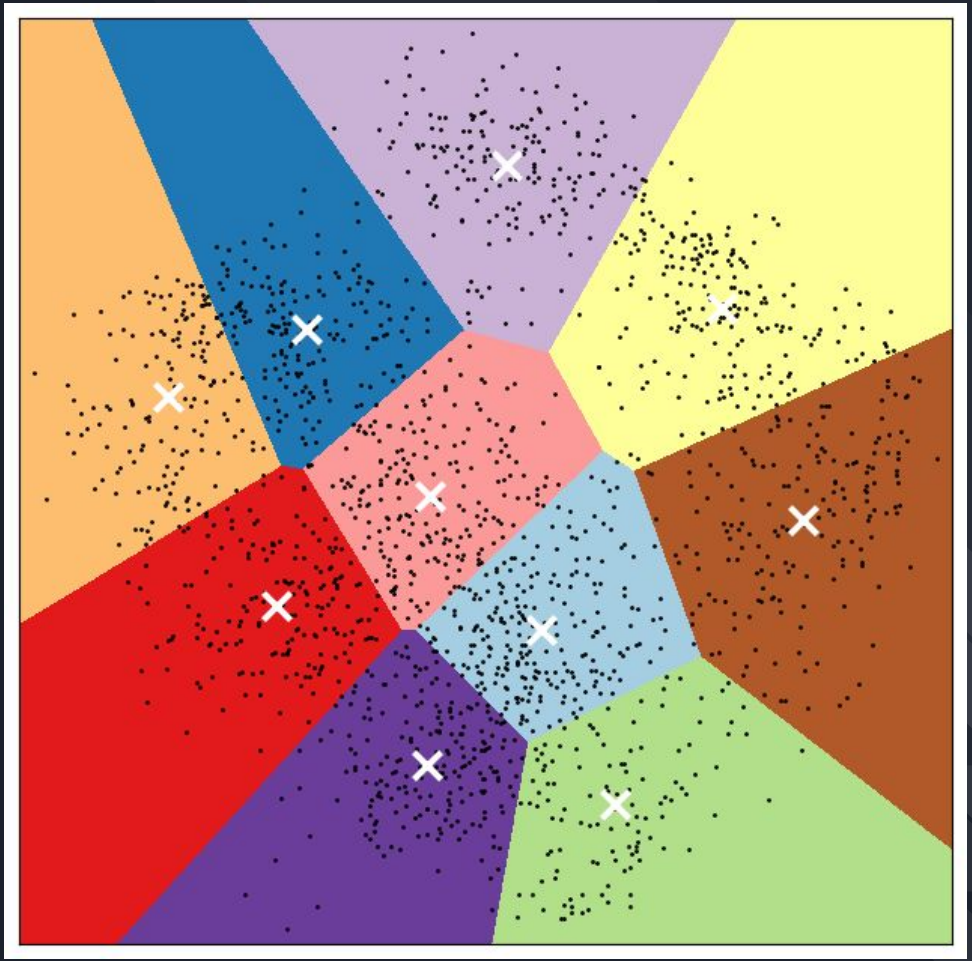
Decision Boundary with Centroids

Here, the MNIST dataset is reduced from 64D to 2D using Principal Component Analysis (PCA) for visualization

Parameters:

- init=random
- n_clusters=10

Inertia: 56387.35



End of Presentation